

AmazonHelp

Evidence-backed AI support automation for noisy, multi-turn customer conversations

Daksh Patel

Final evaluated candidate report

A focused support agent that classifies the issue, remembers what happened, finds relevant historical resolutions, drafts a grounded reply, verifies it, and chooses between automation and human review.

Executive Summary

Customer-support messages are short, noisy, incomplete, and often span several turns. The system therefore treats support as a **decision problem**, not just a text-generation task: understand the problem, preserve context, retrieve evidence, generate conservatively, and stop when risk is too high.

Core principle: maximum automation is not the goal. **Safe, evidence-backed automation is.** If the evidence or policy is insufficient, the correct action is to stop and escalate.

74.5%

ESCALATION F1 · V3

31.4%

UNSAFE AUTO-HANDLE

99%

GROUNDING

0%

CAPABILITY HALLUCINATION

The operating loop



Signal	What the agent asks	Why it matters
Problem	What is the customer's underlying issue?	Determines the right intent and evidence search.
State	What has already happened?	Avoids repetitive questions and wrong next steps.
Evidence	What historical resolution supports the reply?	Reduces generic or invented guidance.
Risk	Is this safe to automate?	Protects sensitive or unresolved cases.

Context first	Evidence first	Risk first
State comes from the live thread, not historical memory.	Historical cases support the answer; they do not become facts about the current customer.	High-risk signals can override the normal generation path.

Project at a glance. 14 frozen intents + conversation state + multi-intent labels + historical Amazon decision points + semantic/lexical RRF + grounding/safety verification + deterministic escalation. V1, V2 and V3 each contain 200 independently evaluated cases.

Candidate thesis: *A strong support agent knows when it has enough evidence to answer and when the correct action is to stop and involve a human.*

Problem Framing & Data Discipline

The Customer Support on Twitter dataset contains roughly 3M noisy, multi-turn tweets across many brands. I selected Amazon and built a focused agent rather than an unconstrained general chatbot. The required behavior is simple to state but difficult to execute reliably: **classify the problem, draft a historically grounded response, and decide whether automation is safe.**

What good support automation means

- Recognize the underlying problem, not just keywords.
- Remember actions already completed in the live conversation.
- Reuse solved cases as evidence rather than generic knowledge.
- Be explicit about capability boundaries.
- Escalate sensitive, unresolved, contradictory, or unsupported cases.

What stays outside the prototype

Direct account/order mutations, payment execution, identity verification, autonomous refunds, unrestricted browsing, and fake handoff execution are excluded. They need real integrations and authorization; pretending otherwise would make evaluation less trustworthy.

A compact support-operations view

Challenge	AmazonHelp mechanism	Outcome
Manual triage	14-intent taxonomy + status	Consistent routing
Fragmented context	Live state + historical retrieval	Fewer repeated questions
Unresolved work	Waiting-state detection	Earlier human ownership
Knowledge buried in cases	Decision-point retrieval	Reuse proven resolutions
Inconsistent replies	Grounding + safety checks	More honest responses

Taxonomy: keep three concepts separate

Concept	Definition	Use
Intent	The underlying problem; 14 frozen leaf intents.	Stable classification surface.
State	What has happened in the current conversation: initial, tracking checked, carrier contacted, details provided, or waiting window exceeded.	Controls next action.
Outcome	How the case ends, including resolved closure.	Separates problem from disposition.
Multi-intent	Multiple problems in one message; not a new combinatorial intent.	Preserves coverage and interpretability.

Representative workflows

Support	Finance	IT
“Still unresolved after three replies.” Preserve history; do not restart the script.	“Invoice is still unpaid after multiple follow-ups.” Repeated failure can require ownership.	“Someone changed my API credential.” Treat as security-sensitive rather than generic reset advice.
Sales	Account management	Logistics
Prior quote + new volume request may need context and routing.	Renewal urgency + repeated unresolved contacts should remain visible.	Carrier incidents require stronger safety coverage than routine tracking questions.

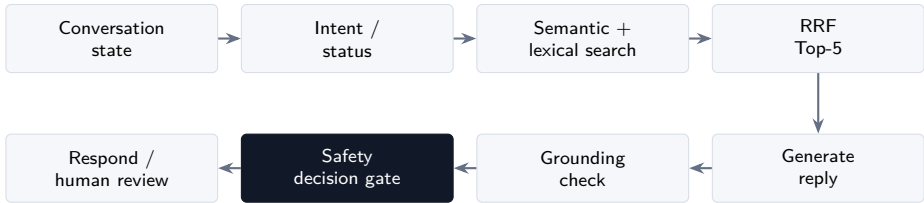
Data controls

167,764 valid retrieval documents after preprocessing. Empty cases were removed. Golden cases were held out before indexing, and retrieval documents preserve provenance so each historical response can be traced back to its source case. The local development index uses a deterministic stratified 10K sample.

Architecture & Safety Design

The design separates **live conversation state** from **historical memory**. A stable `conversation_id` isolates concurrent chats; Qdrant stores the historical Amazon corpus. This makes context ownership explicit and testable.

Observable agent pipeline



System layers

Layer	Implementation	Design reason
Conversation	Recent live turns + extracted state	Preserve prior actions and current problem.
Classification	14 intents + area + multi-label + status	Keep the label space small and measurable.
Retrieval	Customer message + relevant context + historical response	Retrieve concrete precedent.
Reranking	Semantic Top-30 + lexical Top-30 → RRF($k = 60$) → Top-5	Combine meaning with exact support language.
Generation	LLM using conversation + evidence	Draft a context-aware response.
Verification	Grounding, capability, safety	Reject unsupported claims.
Decision	Deterministic high-risk overrides + normal policy	Make safety routing reproducible.
Delivery	Real SSE stream + observable stage events	Keep the live UI transparent without exposing private reasoning.

Safety hierarchy

Account security	Repeated support failure	Carrier misconduct
Compromise, unauthorized changes, takeover signals → human review.	Multiple failed contacts, handoffs, or unresolved journeys → human review.	Refusal, abusive behavior, or package mishandling → human review.

Normal login/password issues remain automatable when no compromise signal exists. Ambiguous requests should trigger clarification. Harmless out-of-scope requests receive a safe decline instead of automatic routing.

Important state rule. The system compares *completed customer actions* with *requested next actions*. “Where can I find the UPS tracking number?” is compatible with having checked delivery status; asking the customer to repeat a completed action is not.

Retrieval mechanics

Step	Mechanism	Purpose
Semantic	all-MiniLM-L6-v2, 384d; Top-30	Capture paraphrases.
Lexical	Top-30 exact/support-language matches	Recover precise entities and wording.
Fusion	RRF: $1/(60 + r_s) + 1/(60 + r_l)$	Combine rankings without raw-score tuning.
Evidence	Top-5 fused cases	Small, inspectable generation context.

Execution and isolation

Each request carries the same `conversation_id` across turns. A second chat receives a separate live state while sharing the historical index. The UI receives observable stages—classification, retrieval, generation, verification, completion—but never private chain-of-thought.

Evaluation honesty. V1–V3 have no gold retrieval-relevance labels. I therefore report RRF candidate instrumentation, not fabricated retrieval recall.

Evaluation & Results

The harness evaluates the system at several levels: human labels for classification/state/escalation, an LLM judge for response quality, claim-level grounding checks, and observable behavioral invariants.

Golden evaluation design

V1	V2	V3
200 cases	200 fresh cases	200 fresh cases
Baseline behavior	First policy iteration	Final policy iteration
No leakage into later sets	New challenge coverage	Zero exact-message leakage vs V1/V2

V3 execution: 200/200 successful · 100% infrastructure success · real backend pipeline · zero mock fallback. The reported candidate is therefore reproducible rather than a manually curated demo.

Headline metrics

Metric	V1	V2	V3
Status accuracy	91.0%	95.5%	95.5%
Primary intent accuracy	74.5%	83.5%	80.5%
Multi-intent exact match	68.0%	81.0%	76.0%
State accuracy	67.0%	84.5%	82.0%
Escalation precision	75.0%	54.1%	81.4%
Escalation recall	40.0%	46.5%	68.6%
Escalation F1	52.2%	50.0%	74.5%
Unsafe auto-handle	60.0%	53.5%	31.4%
Unnecessary escalation	1.08%	10.83%	5.37%
Behavioral policy pass	90.5%	90.5%	93.0%
Grounding	99.5%	99.0%	99.0%
Capability hallucination	0.5%	0%	0%
p50 latency	24.45s	24.85s	22.24s

V3 response quality

Overall	Correctness	Groundedness	Safety / honesty
4.05 / 5	4.44 / 5	4.92 / 5	4.96 / 5

Relevance 4.17 · Actionability 3.63 · Conversation awareness 2.21 · all 200 V3 cases judged.

Baselines

Baseline	Behavior	Purpose
Escalate everything	Every case goes to a human	Safety/recall ceiling; no automation value.
Simple LLM automation	Classification → retrieval → answer	Tests value of explicit state and safety gates.
AmazonHelp V3	State + RAG + RRF + verification + deterministic policy	Balances automation with explicit stop conditions.

The key iteration result. V2 showed the danger of broad overrides: precision fell to 54.1% and unnecessary escalation rose to 10.83%. V3 narrowed the policy and reached 81.4% precision while also increasing recall to 68.6%.

Measurement limits

The LLM judge is a scalable secondary signal, not ground truth. Human labels remain authoritative for classification and escalation. Human judge calibration/agreement should be reported explicitly rather than invented. Retrieval recall also waits for gold relevance labels.

Failure Analysis

The remaining errors are concentrated rather than uniform. That makes the next iteration concrete: improve the semantic safety boundary, not the whole system.

Failure mode	Observed V3 behavior	Next fix
Carrier misconduct	6/13 detected; 46.15% recall. Misses included “chucked the box over the gate” and refusal/insults.	Add semantic safety classification and expand colloquial challenge examples.
Repeated-support journeys	13/19 detected; 68.42% recall. Some users described handoffs instead of contact counts.	Model handoffs, callbacks, appointments and unresolved journeys.
Single-contact confusion	“contacted support yesterday once” could contribute to waiting-state evidence.	Require repeated contact or explicit unresolved failure.
Speaker leakage	“compromised” in a shipping-carton context could inherit account-security routing from prior assistant text.	Scope high-risk detectors to customer-authored turns.
Grounding retry timeout	A small tail of complex cases exhausted grounding retries and safely escalated.	Add an evidence-sufficiency gate and improve retrieval coverage.

Challenge-set snapshot

Carrier misconduct	Repeated support	Security
46.15% recall 100% precision 13 cases	68.42% recall 100% precision 19 cases	84.21% recall 19 cases Lock + failed recovery: 85.71%

Controls held. Routine carrier cases: zero policy false escalations. Ambiguous cases: zero escalation false positives. OOS: zero human-routing false positives after the policy fix. State-consistency rule: zero false-positive escalations.

Safety movement: V1 → V3

Metric	V1	V3	Read
Escalation recall	40.0%	68.6%	More risky cases caught.
Escalation precision	75.0%	81.4%	V3 avoids V2 over-escalation.
Unsafe auto-handle	60.0%	31.4%	16/51 risky cases remain.
Unnecessary escalation	1.08%	5.37%	Safety gains have an operational cost.

What Is Misleading About My Headline Number?

74.5% escalation F1 is an aggregate, not a safety guarantee. V3 still auto-handled 16 of 51 cases expected to escalate. Carrier misconduct recall was only 46.15%.

SAFE ESCALATION

“My account was hacked; someone changed my email and I already tried resetting the password.”

ACCOUNT_SECURITY_COMPROMISE → HUMAN REVIEW

Account-specific intervention is required; generic reset advice is insufficient.

SAFE AUTOMATION

“Where can I find the UPS tracking number?”

ROUTINE CARRIER INSTRUCTION → AUTO-HANDLE

A requested tracking instruction is not a contradiction just because tracking was previously checked.

Interpretation: V3 is the strongest evaluated submission candidate. Its remaining safety gaps are specific, measurable, and directly actionable.

Decisions, Next Week & Submission Readiness

The design favors choices that are measurable, reproducible, and reversible. The log records behavior-changing decisions rather than implementation trivia.

Decision log — 15 choices

Taxonomy	Evidence	Safety
1. Amazon selected for support variety.	6. Retrieve decision points.	9. Deterministic high-risk overrides.
2. Freeze 14 leaf intents.	7. RRF for semantic + lexical fusion.	10. Safe out-of-scope decline.
3. Multi-intent is multi-label.	8. No fabricated retrieval GT.	11. Action-based state consistency.
4. Intent / state / outcome separated.		12. No fake human handoff.
5. State keyed by conversation_id.		13. Reference-free response rubric.
Evaluation	Release	
14. Fresh V2/V3 Golden sets test policy generalization.	15. Freeze V3 as the reproducible submission candidate.	

One-week engineering plan

Safety coverage	Measurement + systems
01 Semantic safety — replace brittle carrier/repeated-support regexes with a small semantic classifier + deterministic thresholds.	04 Retrieval labels — label a relevance subset and measure RRF directly.
02 Speaker awareness — restrict high-risk signals to customer-authored turns.	05 Judge calibration — compare LLM scores with human ratings by dimension.
03 Challenge expansion — add 100–150 colloquial/paraphrased safety cases.	06 Latency — cache embeddings, parallelize retrieval, reduce retries.
	07 Reproducibility — keep V3 frozen while experiments run separately.

Submission checklist

Requirement	Status
Repository	Runnable; pin exact V3 command/environment.
Golden set	200-case V1/V2/V3 evaluation sets.
Evaluation harness	Implemented; V3 200/200 successful.
LLM judge + human agreement	Judge implemented; calibration identified.
Two baselines	All-human ceiling + simple LLM automation.
Failure analysis	Five modes with real examples + hypotheses.
Headline caveat + next-week plan	Explicitly included.
Decision log	15 non-obvious decisions.

Release gates

Freeze evidence	Regression safety	Autonomy gate
Keep V1/V2/V3 immutable; record IDs and leakage checks.	Retain security, carrier, repeated-support, state, ambiguity and OOS tests.	Do not expand autonomy until unsafe auto-handle falls materially and safety recall improves.

Final candidate positioning. This submission demonstrates end-to-end ownership: noisy-data selection, taxonomy, conversational state, RAG, reranking, safety policy, evaluation, failure analysis, and reproducibility.

The strongest story is not “My chatbot got X%.” It is: “I built a system that knows when it can safely automate, proved where it fails, and designed the next experiment from those failures.”